

Taller Angular: Integración de APIs, Formularios y Asociaciones

Preguntas conceptuales:

1. Sobre Observables y Asincronía:

- a. ¿Por qué las peticiones *HTTP* en Angular devuelven un *Observable* en lugar de la data directa? ¿Qué diferencia hay frente a una *Promise*?

Rta: Las peticiones *HTTP* en Angular devuelven un *Observable* porque las respuestas llegan de manera asincrona. Este mismo permite esperar la respuesta sin bloquear la aplicación y además ofrece herramientas para manejar multiples valores en el tiempo.

La diferencia frente a una *Promise* es que una *Promise* solo devuelve un único valor y se ejecuta inmediatamente, mientras que un *Observable* puede emitir varios valores y solo se ejecuta cuando se hace *subscribe()*.

- b. ¿Qué ocurre si un componente llama a un servicio *HTTP* pero nunca hace *subscribe()*? ¿Se ejecuta la petición igualmente?

Rta: Si un componente llama a un servicio *HTTP* pero nunca hace *subscribe()*, la petición no se ejecuta. En Angular, los Observables son “*lazy*”, es decir, quedan preparados, pero no hacen nada hasta que alguien se suscribe.

2. Sobre Inyección de Dependencias:

- c. ¿Qué ventaja de modularidad da *providedIn: 'root'* frente a instanciar el servicio manualmente con *new CityService()* dentro de un componente?

Rta: La ventaja de usar *providedIn: 'root'* es que Angular crea una unica instancia compartida del servicio para toda la aplicación, facilitando la reutilización y el mantenimiento. Además permite que Angular maneje automáticamente la inyección de dependencias, en lugar de crear objetos manualmente con *new*.

- d. ¿Cómo facilitaría la inyección de dependencias la escritura de pruebas unitarias para estos servicios?

Rta: La inyección de dependencias facilita las pruebas unitarias porque permite reemplazar servicios reales por versiones simuladas. De esta manera se pueden probar componentes y servicios de forma aislada sin depender de *APIs* reales o bases de datos.

3. Sobre Interceptores:

- e. ¿Por qué es mejor centralizar el manejo de errores en *HttpInterceptor* en lugar de poner bloques try/catch en cada componente?

Rta: Es mejor centralizar el manejo de errores en un *HttpInterceptor* porque evita repetir el mismo código en cada componente y mantiene la aplicación más organizada. Además cualquier cambio en el manejo de errores se realiza en un solo lugar.

- f. Además del manejo de errores, mencione dos casos de uso adicionales para un interceptor.

Rta:

- Agregar automáticamente tokens de autenticación a las peticiones *HTTP*.
- Registrar o medir información de las peticiones y respuestas, como tiempos de respuesta o logs.

4. Sobre el Patrón Maestro-Detalle:

- g. ¿Por qué se usa *@Input()* para pasar la ciudad a *CityDetailComponent* en lugar de volver a hacer un *GET* a */api/cities/{id}*?

Rta: Se usa *@Input()* para pasar la ciudad a *CityDetailComponent* porque la ciudad ya fue cargada en la lista principal. Así se evita hacer otra petición innecesaria al *backend* y se aprovecha la información que ya tiene el componente padre.

- h. Observe que en la respuesta de *GET /api/cities*, el campo *country* ya viene como objeto anidado (no solo el *countryId*). ¿Qué ventaja de diseño tiene esta decisión del *backend* para el *frontend*?

Rta: La ventaja de que *country* venga como objeto anidado es que el *frontend* ya recibe la información completa del país junto con la ciudad. Esto facilita mostrar datos como el nombre del país sin tener que hacer otra petición usando el *countryId*.